

# Enterprise Blockchain Case Studies

TypeScript repository for enterprise blockchain design, protocol mapping, and integration boundaries.

## Scope

- Case studies tied to operating problems
- Protocol-specific transaction models
- SDK-oriented integration patterns

# Repository Structure

| Area       | Purpose                                                  |
|------------|----------------------------------------------------------|
| modules/   | Domain logic, protocol adapters, and integration clients |
| examples/  | Runnable case studies and protocol projections           |
| contracts/ | Solidity contract source and ABI artifact                |
| docs/      | Research, architecture notes, and presentation material  |

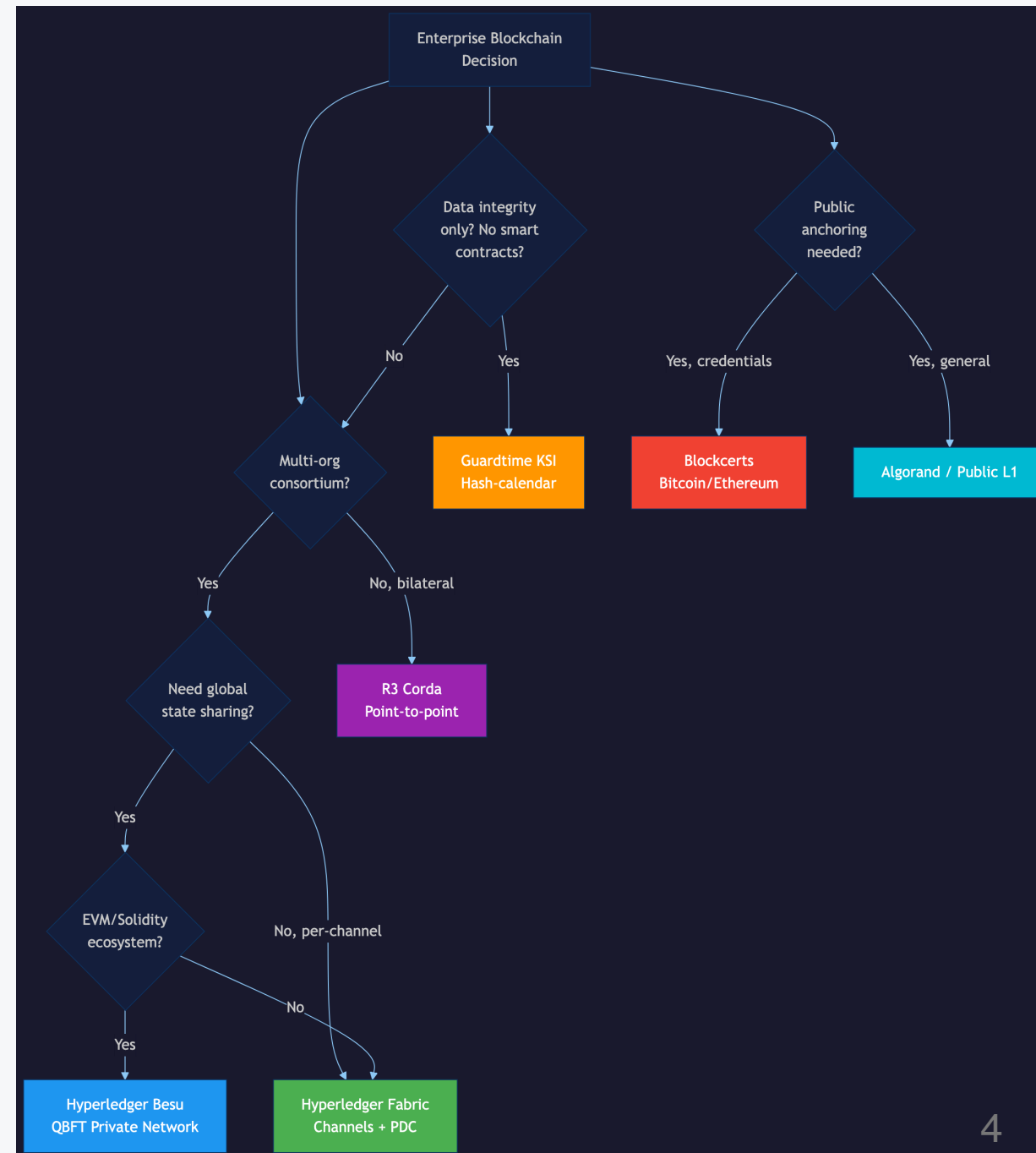
# Operating Problems Covered

1. Food recall response
2. Consortium order sharing
3. Hospital staffing clearance
4. Aid voucher reconciliation

These scenarios were selected because they require provenance, privacy, cross-organization coordination, or settlement controls.

# Architecture Model

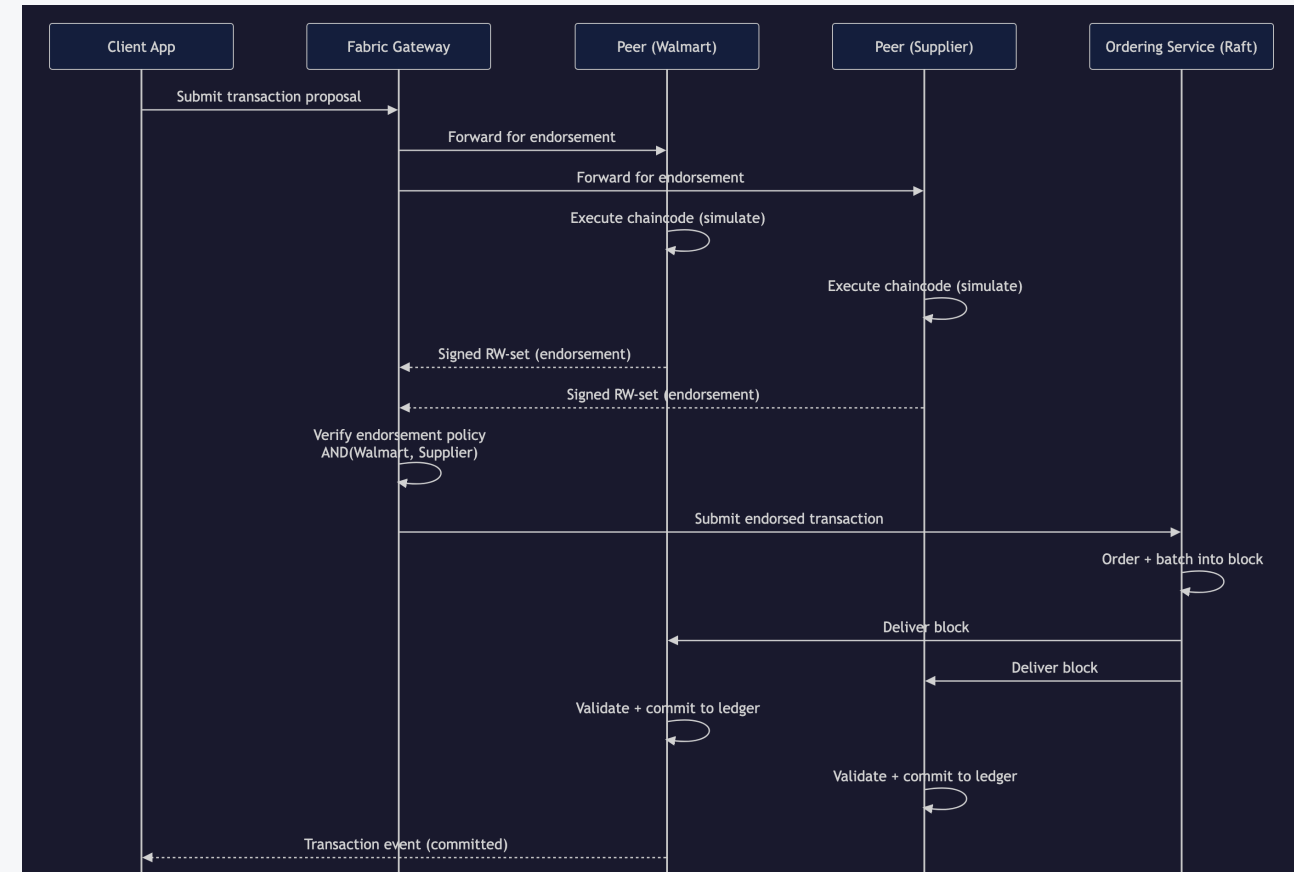
- Domain modules hold the business rules.
- Protocol adapters capture platform semantics.
- Integration clients shape runtime requests.
- Environment configuration supplies operational inputs.



# Food Recall Response

## Why it matters

- Rapid impact analysis during a recall
- Shared traceability across supplier, carrier, and retailer boundaries
- Deterministic commit model for operational response



# Selective Disclosure

## Key point

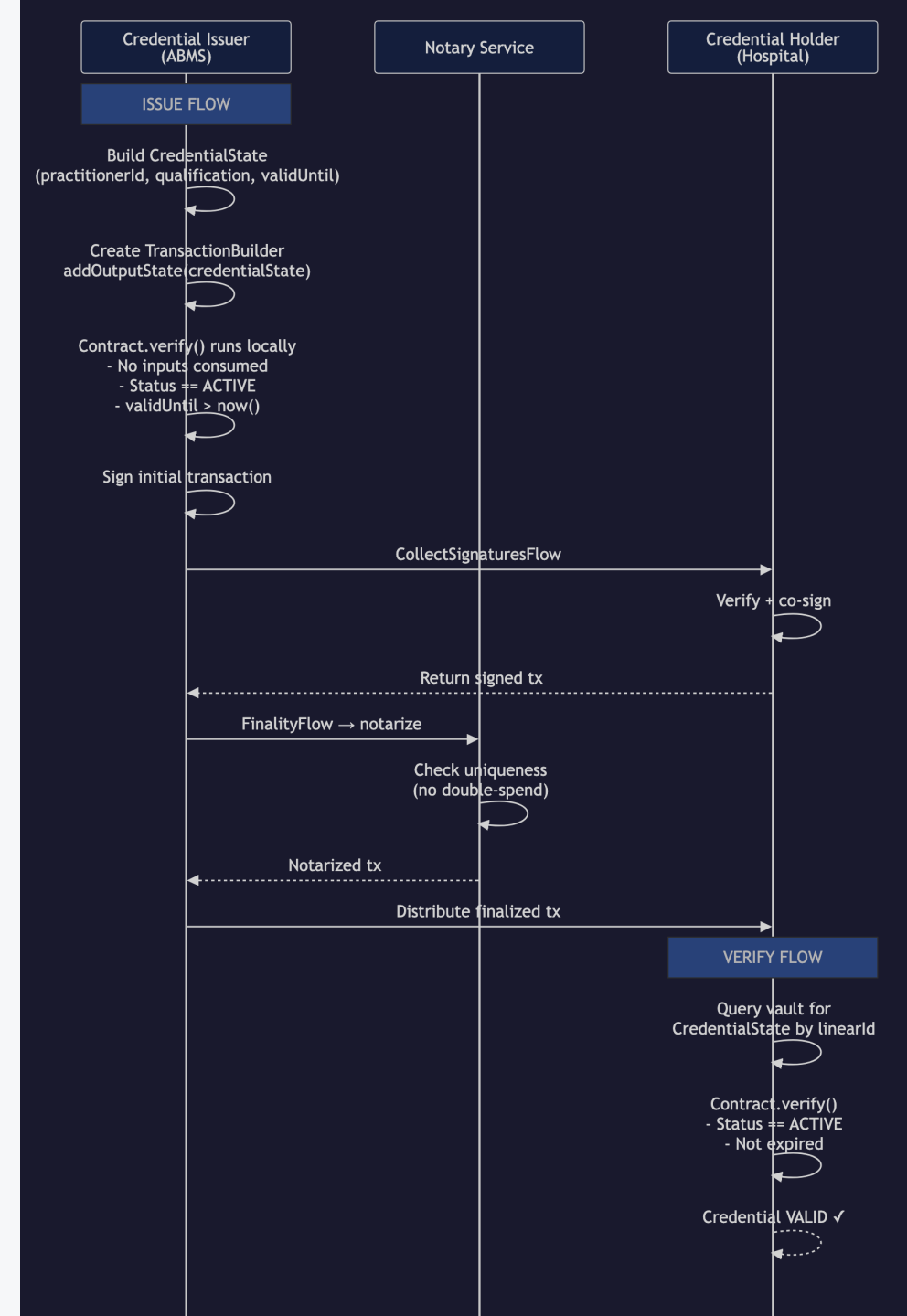
The same commercial record can be anchored once and disclosed differently to logistics, banking, regulatory, and supplier audiences.



# Credential Verification

## Key point

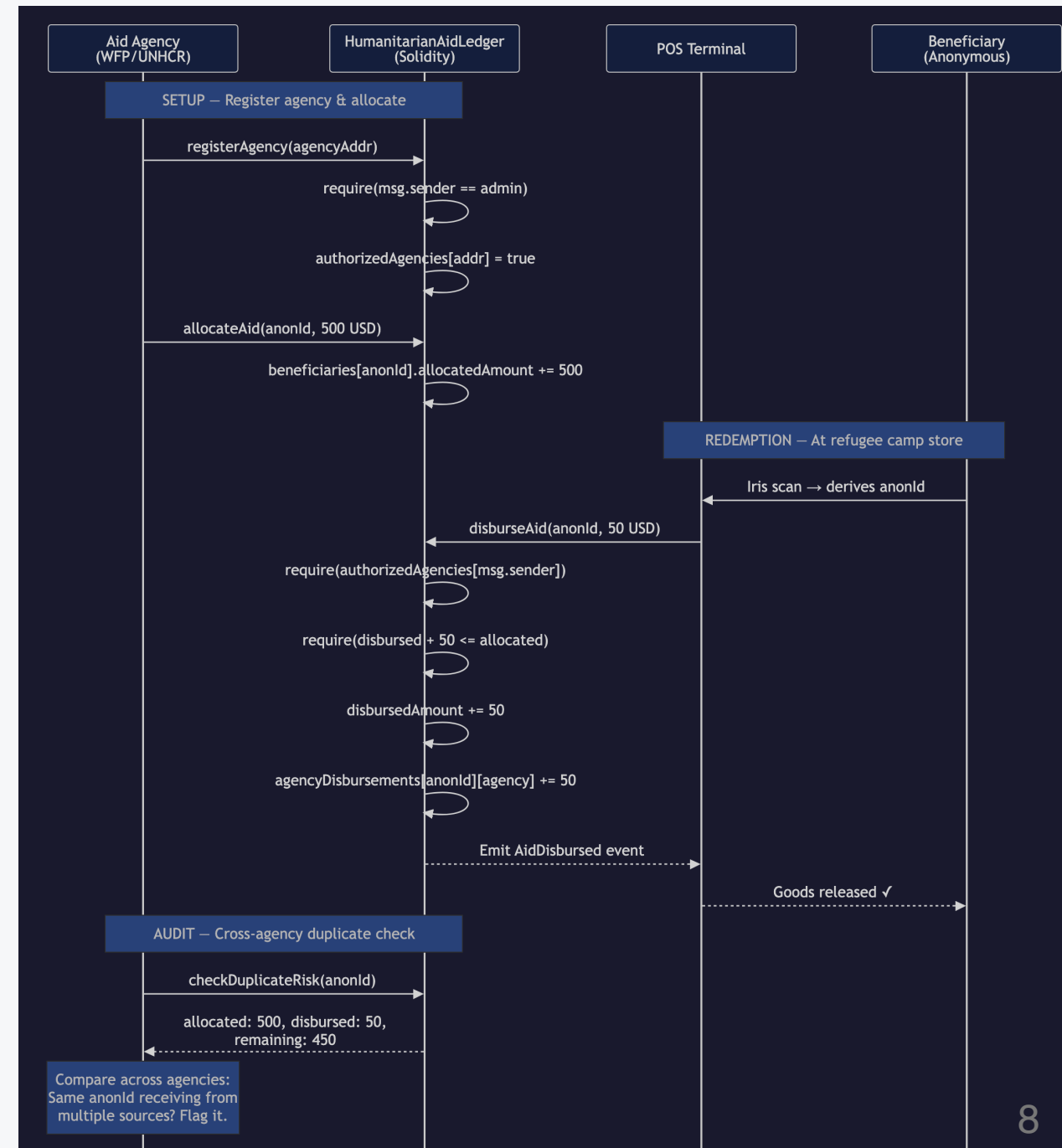
Need-to-know distribution is often a better fit than broad ledger replication for regulated staffing workflows.



# Reconciliation And Controls

## Key point

Shared ledgers matter when multiple organizations need one settlement view without delegating control to a single operator.





# Platform Fit

Decision criteria in this repository:

- Governance model
- Privacy boundary
- Finality model
- Integration surface



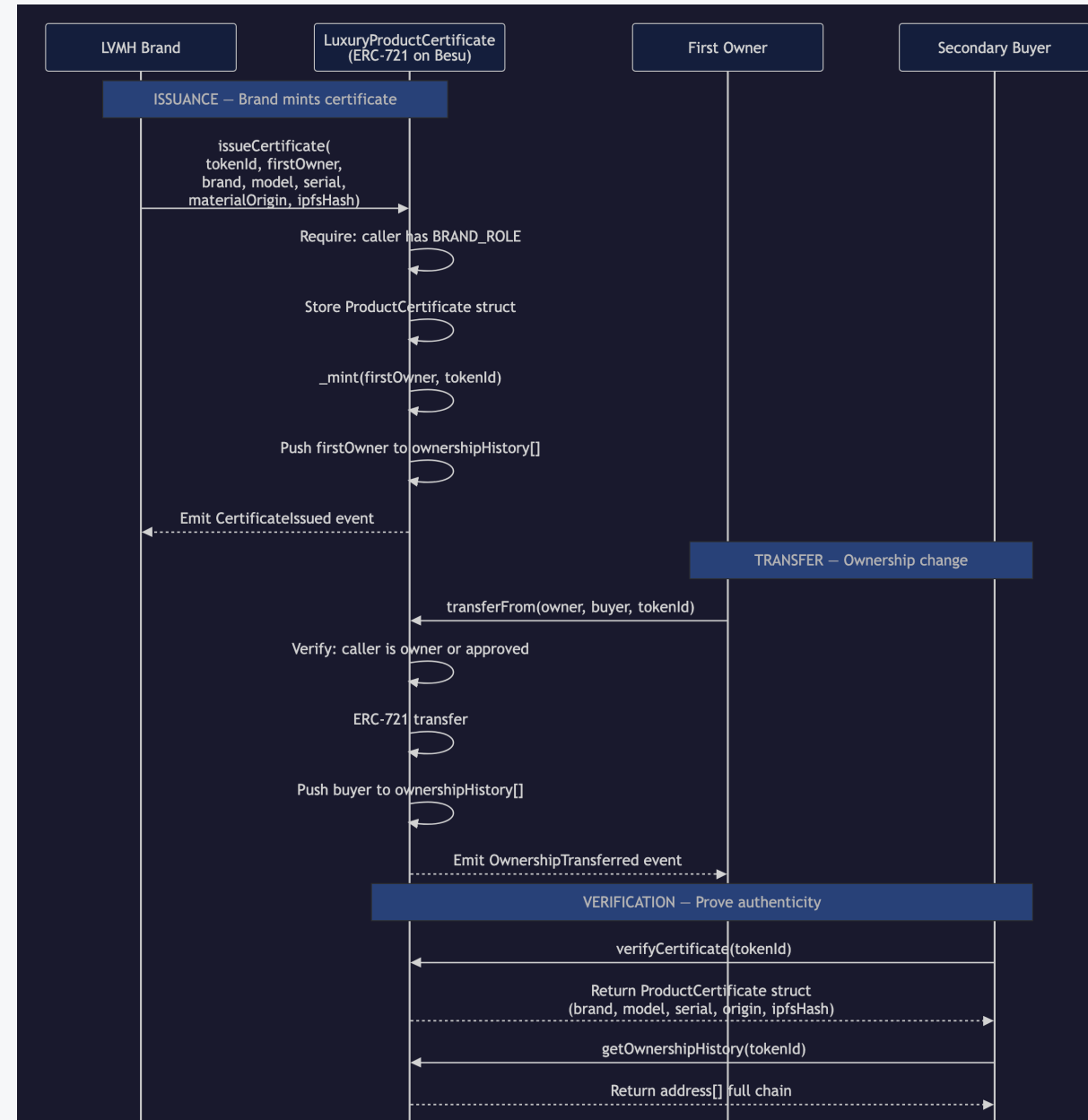
# From Domain Logic To Runtime

1. Domain module defines the business rule.
2. Protocol adapter maps the event into a platform-specific transaction.
3. Integration client shapes the request for the runtime boundary.
4. Environment configuration supplies credentials, endpoints, and contract addresses.

# Besu Path

Repository assets:

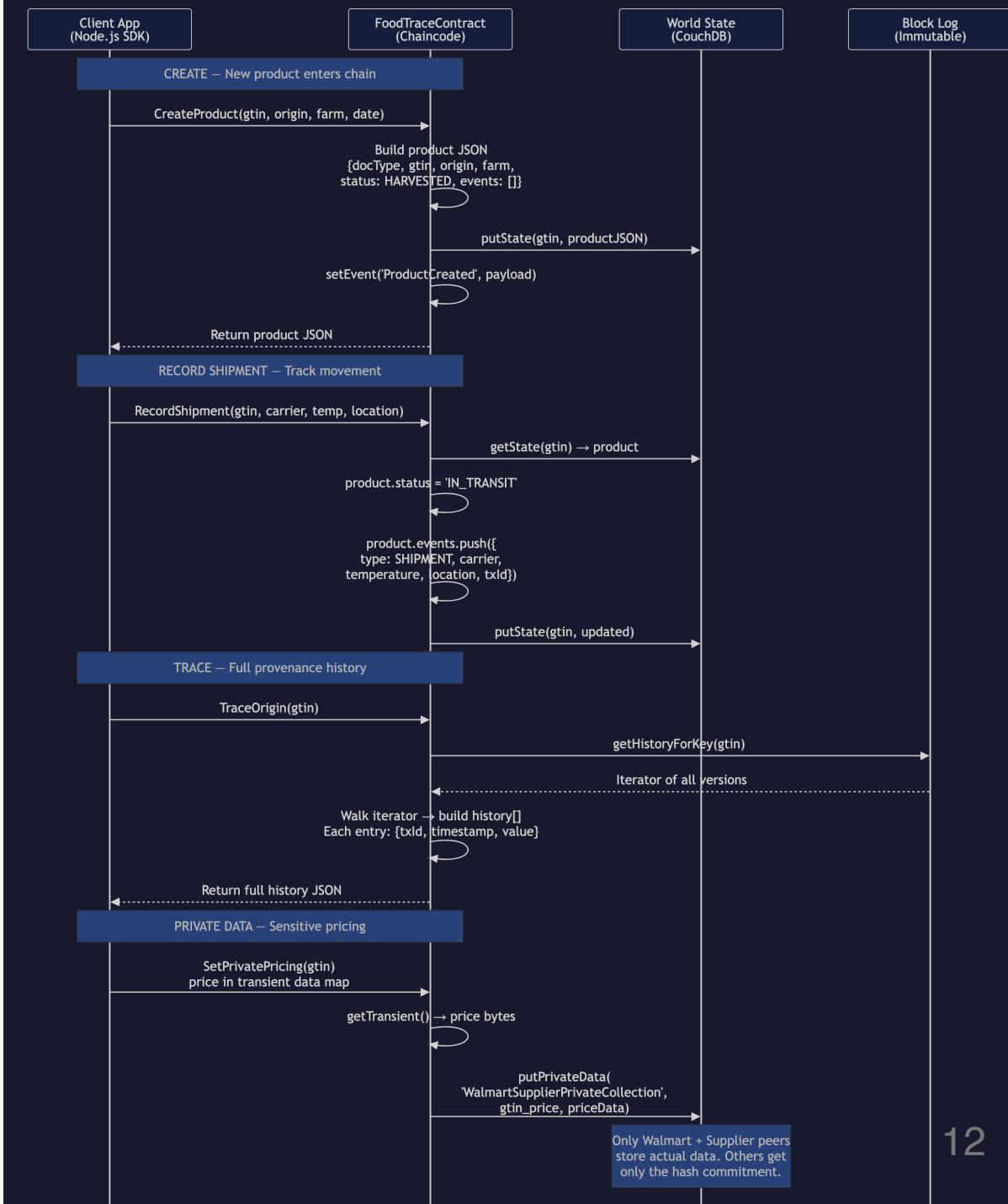
- Solidity source in  
`contracts/ConsortiumOrderRegistry.sol`
- ABI artifact in  
`contracts/ConsortiumOrderRegistry.json`
- ethers integration in  
`modules/integrations/besu-client/`



# Fabric Path

Repository assets:

- Proposal builders in `modules/protocols/fabric/`
- Gateway integration in `modules/integrations/fabric-gateway/`
- Environment template in `examples/config/fabric.env.example`



# Corda Path

TypeScript is the integration layer, not the CorDapp runtime.

Repository assets:

- Clearance flow projection in `modules/protocols/corda/`
- Gateway request builder in `modules/integrations/corda-gateway/`
- Environment template in `examples/config/corda.env.example`

# How To Use The Repository

```
npm install  
npm run verify  
npm run demo:adapters  
npm run demo:integrations
```

Then move from `examples/` to `modules/` to `docs/architecture/`.